PRACTICAL - 01

```
#include <iostream>

#include <vector>

#include <chrono>


using namespace std;

using namespace std::chrono;


// Linear Search Function

int linearSearch(const vector<int>& arr, int key)

{

   for (int i = 0; i < arr.size(); i++)

   {

      if (arr[i] == key)

         return i;

   }

   return -1;

}


// Binary Search Function

int binarySearch(const vector<int>& arr, int key)

{

   int low = 0;

   int high = arr.size() - 1;


   while (low <= high)

   {

      int mid = low + (high - low) / 2;
```

```
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    int n;
    cout << "Enter the size of the array: ";
    cin >> n;

    vector<int> arr(n);

    // Create a sorted array
    for (int i = 0; i < n; i++)
        arr[i] = i + 1;

    int key;
    cout << "Enter the element to search: ";
    cin >> key;
```

```
if (key < 1 || key > n)
{
   cout << "\nElement is outside the array range.\n";
   return 0;
}

int index;

// Linear Search
auto start = high_resolution_clock::now();
index = linearSearch(arr, key);
auto stop = high_resolution_clock::now();

auto linearTime = duration_cast<microseconds>(stop - start);

cout << "\n===== Linear Search =====\n";
if (index != -1)
   cout << "Element found at index: " << index << endl;
else
   cout << "Element not found." << endl;

cout << "Execution Time: " << linearTime.count()
    << " microseconds\n";

// Binary Search
start = high_resolution_clock::now();
index = binarySearch(arr, key);
stop = high_resolution_clock::now();
```

```
    auto binaryTime = duration_cast<microseconds>(stop - start);

    cout << "\n===== Binary Search =====\n";
    if (index != -1)
        cout << "Element found at index: " << index << endl;
    else
        cout << "Element not found." << endl;

    cout << "Execution Time: " << binaryTime.count()
         << " microseconds\n";

    // Comparison
    cout << "\n===== Performance Comparison =====\n";
    if (linearTime > binaryTime)
        cout << "Binary Search is faster than Linear Search.\n";
    else if (linearTime < binaryTime)
        cout << "Linear Search is faster than Binary Search.\n";
    else
        cout << "Both searches took the same time.\n";

    return 0;
}
```